S02 – Bases de Datos Relacionales en MySQL

Evidencias de ejecución – IT Academy, Data Analytics

Nombre	*Carlos Hisamitsu*
P2P	*Camila*
Fecha	*21/09/2026*
Repositorio GitHub	https://github.com/chisamitsu/ita-dataanalytics/tree/main/s02-bases-de-datos-relacionales-mysql

NIVEL 1

Ejercicio 1 – Importación de tablas y diagrama entidad-relación

Enunciado

A partir de los documentos adjuntos (estructura_dades y dades_introduir), importa las dos tablas. Muestra las características principales del esquema creado y explica las diferentes tablas y variables que existen. Asegúrate de incluir un diagrama que ilustre la relación entre las diferentes tablas y variables.

Evidencia de ejecución (Workbench)

```sql
-- Creamos la base de datos
CREATE DATABASE IF NOT EXISTS transactions;
USE transactions;

-- Creamos la tabla company
CREATE TABLE IF NOT EXISTS company (
    id VARCHAR(15) PRIMARY KEY,
    company_name VARCHAR(255),
    phone VARCHAR(15),
    email VARCHAR(100),
    country VARCHAR(100),
    website VARCHAR(255)
);

-- Creamos la tabla transaction
CREATE TABLE IF NOT EXISTS transaction (
    id VARCHAR(255) PRIMARY KEY,
    credit_card_id VARCHAR(15) REFERENCES credit_card(id),
    company_id VARCHAR(20),
    user_id INT REFERENCES user(id),
    lat FLOAT,
    longitude FLOAT,
    timestamp TIMESTAMP,
    amount DECIMAL(10, 2),
    declined BOOLEAN,
```

Output

Action Output

#	Time	Action	Message
1	12:54:28	DROP DATABASE `transactions`	2 row(s) affected
2	12:54:31	CREATE DATABASE IF NOT EXISTS transactions	1 row(s) affected
3	12:54:31	USE transactions	0 row(s) affected
4	12:54:31	CREATE TABLE IF NOT EXISTS company (id VARCHAR(15) PRIMARY KEY, company_name VARCHAR(2...	0 row(s) affected
5	12:54:31	CREATE TABLE IF NOT EXISTS transaction (id VARCHAR(255) PRIMARY KEY, credit_card_id VARCHAR(...	0 row(s) affected

```
INSERT INTO transaction (id, credit_card_id, company_id, user_id, lat, longitude, timestamp, amount, declined) VALUES ('DF27AB
INSERT INTO transaction (id, credit_card_id, company_id, user_id, lat, longitude, timestamp, amount, declined) VALUES ('5A9808
INSERT INTO transaction (id, credit_card_id, company_id, user_id, lat, longitude, timestamp, amount, declined) VALUES ('FBC785
INSERT INTO transaction (id, credit_card_id, company_id, user_id, lat, longitude, timestamp, amount, declined) VALUES ('16C3BD
INSERT INTO transaction (id, credit_card_id, company_id, user_id, lat, longitude, timestamp, amount, declined) VALUES ('3D39E0
INSERT INTO transaction (id, credit_card_id, company_id, user_id, lat, longitude, timestamp, amount, declined) VALUES ('E18311
INSERT INTO transaction (id, credit_card_id, company_id, user_id, lat, longitude, timestamp, amount, declined) VALUES ('65FF0A
INSERT INTO transaction (id, credit_card_id, company_id, user_id, lat, longitude, timestamp, amount, declined) VALUES ('D7AE2C
INSERT INTO transaction (id, credit_card_id, company_id, user_id, lat, longitude, timestamp, amount, declined) VALUES ('C312BA
INSERT INTO transaction (id, credit_card_id, company_id, user_id, lat, longitude, timestamp, amount, declined) VALUES ('3EFB19
```

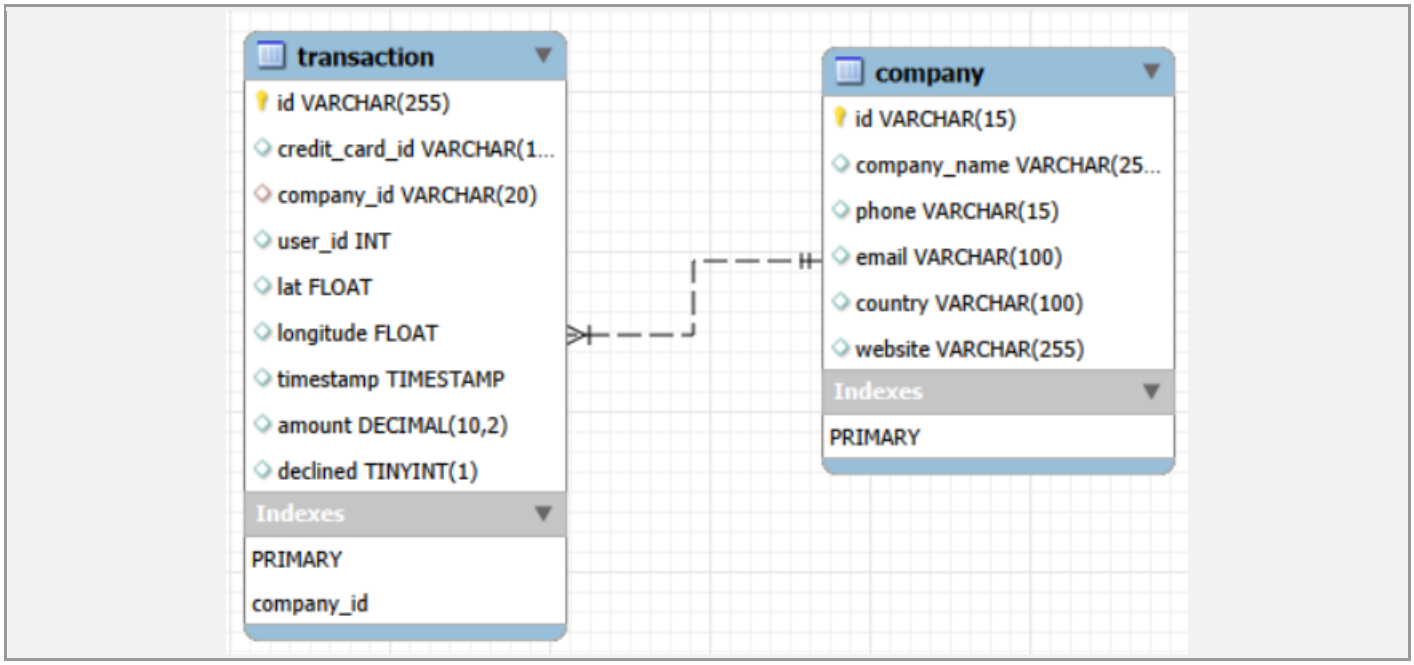
Output

Action Output

#	Time	Action	Message
100097	13:03:27	INSERT INTO transaction (id, credit_card_id, company_id, user_id, lat, longitude, timestamp, amount, decli...	1 row(s) affected
100098	13:03:27	INSERT INTO transaction (id, credit_card_id, company_id, user_id, lat, longitude, timestamp, amount, decli...	1 row(s) affected
100099	13:03:27	INSERT INTO transaction (id, credit_card_id, company_id, user_id, lat, longitude, timestamp, amount, decli...	1 row(s) affected
100100	13:03:27	INSERT INTO transaction (id, credit_card_id, company_id, user_id, lat, longitude, timestamp, amount, decli...	1 row(s) affected
100101	13:03:27	INSERT INTO transaction (id, credit_card_id, company_id, user_id, lat, longitude, timestamp, amount, decli...	1 row(s) affected

Diagrama entidad-relación

Diagrama exportado desde Workbench (Database > Reverse Engineer):



Hay dos tablas en esa base de datos:

1. company (dimensiones)
   Cada fila tiene los datos de una compañía como nombre, contacto (phone, email) y país y sitio web.
   Hay una **clave primaria** en el "id" de la compañía.
2. transaction (hechos)
   Cada fila tiene información de las transacciones realizadas (o rechazadas). Datos de tarjeta de crédito, localidad, fecha y hora, valor, el usuario que lo ha hecho y a que compañía está relacionada.
   Hay una **clave primaria** "id" y una **llave foránea** "company_id" que hace la relación con la tabla company.

Ejercicio 2 – Consultas con JOIN

Enunciado

Utilizando JOIN realizarás las siguientes consultas: listado de los países que están generando ventas; desde cuántos países se generan las ventas; identifica la compañía con la media más grande de ventas.

Evidencia de ejecución (Workbench)

```
26    -- --- Ejercicio 2 ----------------------------------------------------------
27    -- Utilizando JOIN realizarás las siguientes consultas:
28    -- - Listado de los países que están generando ventas.
29    SELECT DISTINCT `company`.`country`
30    FROM `company`
31    INNER JOIN `transaction` ON `transaction`.`company_id` = `company`.`id`
32    WHERE `transaction`.`declined` = 0;
33
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

country
Germany
Australia
United States
New Zealand
Norway
United Kingdom
Italy

Result 2 | Result 3 | Result 4 | transaction 5 | company 6 | company 7 | company 8

Output

Action Output

#	Time	Action	Message
1	11:22:46	USE transactions	0 row(s) affected
2	11:22:46	SELECT DISTINCT `company`.`country` FROM `company` INNER JOIN `transaction` ON `transaction`.`co...	15 row(s) returned

```
34    -- - Desde cuántos países se generan las ventas.
35    SELECT COUNT(DISTINCT `company`.`country`) AS `countries_with_sales`
36    FROM `company`
37    INNER JOIN `transaction` ON `transaction`.`company_id` = `company`.`id`
38    WHERE `transaction`.`declined` = 0;
39
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

countries_with_sales
15

Result 2 | Result 3 | Result 4 | transaction 5 | company 6 | company 7 | company 8

Output

Action Output

#	Time	Action	Message
3	11:22:46	SELECT COUNT(DISTINCT `company`.`country`) AS `countries_with_sales` FROM `company` INNER JOI...	1 row(s) returned

```sql
-- - Identifica la compañía con la media más grande de ventas.
SELECT `company`.`id`,
       `company`.`company_name`,
       ROUND(AVG(`transaction`.`amount`),2) AS `average_sales`
FROM `company`
INNER JOIN `transaction` ON `transaction`.`company_id` = `company`.`id`
WHERE `transaction`.`declined` = 0
GROUP BY `company`.`id`
ORDER BY AVG(`transaction`.`amount`) DESC
LIMIT 1;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

id	company_name	average_sales
b-2222	Ac Fermentum Incorporated	284.91

Result 9

Output

Action Output

#	Time	Action	Message
1	11:29:57	SELECT `company`.`id`, `company`.`company_name`, ROUND(AVG(`transaction`.`amount`),2) AS `aver...	1 row(s) returned

Ejercicio 3 – Subconsultas (sin JOIN)

Enunciado

Utilizando únicamente subconsultas (sin utilizar JOIN): muestra todas las transacciones realizadas por empresas de Alemania; lista las empresas que han realizado transacciones por un amount superior a la media de todas las transacciones; eliminarán del sistema las empresas que no tienen transacciones registradas, entrega el listado de estas empresas.

Evidencia de ejecución (Workbench)

```sql
-- --- Ejercicio 3 ----------------------------------------------------------
-- Utilizando únicamente subconsultas (sin utilizar JOIN):
-- - Muestra todas las transacciones realizadas por empresas de Alemania.
SELECT *
FROM `transaction`
WHERE `transaction`.`company_id` IN (SELECT `id`
                                     FROM `company`
                                     WHERE `country` = "Germany");
```

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content: | Fetch rows:

id	credit_card_id	company_id	user_id	lat	longitude	timestamp	amount	declined
00138D3B-206D-4C03-94B7-63A2676EB9B4	CcS-4899	b-2222	318	41.3781	12.447	2020-03-25 10:43:43	426.36	0
0013C1B6-3B84-4D6C-8154-E2B3FEBCA8E9	CcS-5070	b-2222	489	41.3814	2.18176	2020-12-17 18:15:37	316.90	0
00201A11-2E62-44C4-941D-198FC8DB77F0	CcU-3512	b-2222	193	55.5704	-3.65129	2021-01-22 23:44:27	453.04	0
00235618-0A5C-4D49-9DCB-B3A9405D8923	CcS-8137	b-2222	3556	59.8421	18.729	2020-09-09 15:43:19	263.14	0
005A5A7B-1F1A-4B6C-9B15-1625A78C9C38	CcS-8998	b-2222	4417	41.1591	-8.63905	2024-05-15 09:10:11	442.01	0

transaction 48

Output

Action Output

#	Time	Action	Message
1	13:36:03	SELECT * FROM `transaction` WHERE `transaction`.`company_id` IN (SELECT `id` FROM `company` WHE...	13291 row(s) returned

```
55      -- - Lista las empresas que han realizado transacciones por un amount
56      --   superior a la media de todas las transacciones.
57  •   SELECT *
58      FROM `company`
59      WHERE `company`.`id` IN (SELECT DISTINCT(`company_id`)
60                               FROM `transaction`
61                               WHERE `amount` > (SELECT AVG(`amount`) FROM `transaction`));
```

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content:

id	company_name	phone	email	country	website
b-2222	Ac Fermentum Incorporated	06 85 56 52 33	donec.porttitor.tellus@yahoo.net	Germany	https://instagram.com/site
b-2226	Magna A Neque Industries	04 14 44 64 62	risus.donec.nibh@icloud.org	Australia	https://whatsapp.com/group/9
b-2230	Fusce Corp.	08 14 97 58 85	risus@protonmail.edu	United States	https://pinterest.com/sub/cars
b-2234	Convallis In Incorporated	06 66 57 29 50	mauris.ut@aol.couk	Germany	https://cnn.com/user/110
b-2238	Ante Iaculis Nec Foundation	08 23 04 99 53	sed.dictum.proin@outlook.ca	New Zealand	https://netflix.com/settings

company 49

Output

Action Output

#	Time	Action	Message
1	13:38:30	SELECT * FROM `company` WHERE `company`.`id` IN (SELECT DISTINCT(`company_id`) FROM `transacti...	100 row(s) returned

```
63      -- - Eliminarán del sistema las empresas que no tienen transacciones
64      --   registradas, entrega el listado de estas empresas.
65  •   SELECT *
66      FROM `company`
67      WHERE `company`.`id` NOT IN (SELECT DISTINCT(`company_id`)
68                                   FROM `transaction`);
```

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content:

id	company_name	phone	email	country	website
NULL	NULL	NULL	NULL	NULL	NULL

company 50

Output

Action Output

#	Time	Action	Message
1	13:39:56	SELECT * FROM `company` WHERE `company`.`id` NOT IN (SELECT DISTINCT(`company_id`) FROM `tra...	0 row(s) returned

Comentarios sobre los resultados

1. Hay un total de 100 compañías, y el resultado de la segunda consulta tiene todas las compañías debido a que cada compañía tiene por lo menos una transacción de valor más alto que la media de todas las transacciones.
2. No hay compañía sin transacciones en esa base de datos. Por eso la tercera consulta no retornó resultado alguno.

Ejercicio 4 – Creación de la tabla credit_card

Enunciado

Diseña y crea una tabla llamada "credit_card" que almacene detalles cruciales sobre las tarjetas de crédito. Debe identificar de manera única cada tarjeta y relacionarse adecuadamente con "transaction" y "company". Tras crearla, ingresa la información del documento "dades_introduir_credit". Muestra el diagrama y una breve descripción.

Evidencia de ejecución (Workbench)

```sql
-- --- Ejercicio 4 -------------------------------------------------------
-- Tu tarea es diseñar y crear una tabla llamada "credit_card" que
-- almacene detalles cruciales sobre las tarjetas de crédito. La nueva
-- tabla debe ser capaz de identificar de manera única cada tarjeta y
-- establecer una relación adecuada con las otras dos tablas
-- ("transaction" y "company"). Después de crear la tabla será necesario
-- que ingreses la información del documento denominado
-- "dades_introduir_credit". Recuerda mostrar el diagrama y realizar una
-- breve descripción de este.
-- (datos -> recursos/N1-Ex.4__dades_introduir_credit.sql)
CREATE TABLE `credit_card` (
    `id` VARCHAR(15) PRIMARY KEY,
    `iban` VARCHAR(34) NOT NULL,
    `pan` VARCHAR(19) NOT NULL,
    `pin` VARCHAR(4),
    `cvv` VARCHAR(3),
    `expiring_date` VARCHAR(8)
);
```

Output

Action Output

#	Time	Action	Message
1	12:54:50	CREATE TABLE `credit_card` (`id` VARCHAR(15) PRIMARY KEY, `iban` VARCHAR(34) NOT NULL, `p...	0 row(s) affected

```sql
-- Insertamos datos de credit_card
INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('CcU-2938', 'TR3019503
INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('CcU-2945', 'DO2685476
INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('CcU-2952', 'BG45IVQL5
INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('CcU-2959', 'CR7242477
INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('CcU-2966', 'BG72LKTQ1
INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('CcU-2973', 'PT8780622
INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('CcU-2980', 'DE3924188
INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('CcU-2987', 'GE8968143
INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('CcU-2994', 'BH6271442
INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('CcU-3001', 'CY4908742
INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('CcU-3008', 'LU5072166
INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('CcU-3015', 'PS1193982
INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('CcU-3022', 'GT9169516
INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('CcU-3029', 'AZ6231741
INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('CcU-3036', 'AZ3933600
INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('CcU-3043', 'TN6488143
INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('CcU-3050', 'FR5167744
INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('CcU-3057', 'LU9318225
INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('CcU-3064', 'PS1469655
INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('CcU-3071', 'NO8923814
INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('CcU-3078', 'IS0251271
INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('CcU-3085', 'BE6311472
INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('CcU-3092', 'RO65LSOD1
```

Output

Action Output

#	Time	Action	Message
4995	12:55:49	INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('CcS-9576', 'XX529710930587...	1 row(s) affected
4996	12:55:49	INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('CcS-9577', 'XX158914078594...	1 row(s) affected
4997	12:55:49	INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('CcS-9578', 'XX991539646456...	1 row(s) affected
4998	12:55:49	INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('CcS-9579', 'XX296393091587...	1 row(s) affected
4999	12:55:49	INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('CcS-9580', 'XX781258889851...	1 row(s) affected
5000	12:55:49	INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES ('CcS-9581', 'XX915670516405...	1 row(s) affected

```sql
-- El archivo dades_introduir_credit carga las fechas en formato mm/dd/yy;
-- hay que convertirlas al formato de fecha de MySQL.
-- NOTA: el LIMIT (igual al total de filas de la tabla) es necesario para
-- pasar el modo seguro (SQL_SAFE_UPDATES) sin tener que desactivarlo,
-- ya que esta actualizacion no tiene una condicion WHERE restrictiva
-- (se aplica a todas las filas).
UPDATE credit_card
SET expiring_date = STR_TO_DATE(expiring_date, '%m/%d/%y')
LIMIT 5000;
```

Output

Action Output

#	Time	Action	Message
1	13:26:49	UPDATE credit_card SET expiring_date = STR_TO_DATE(expiring_date, '%m/%d/%y') LIMIT 5000	5000 row(s) affected Rows matched: 5000 Changed: 5000 Warnings: 0

```sql
-- Cambiamos el tipo de la columna a DATE
ALTER TABLE credit_card
MODIFY expiring_date DATE;
```

Output

Action Output

#	Time	Action
1	13:29:03	ALTER TABLE credit_card MODIFY expiring_date DATE

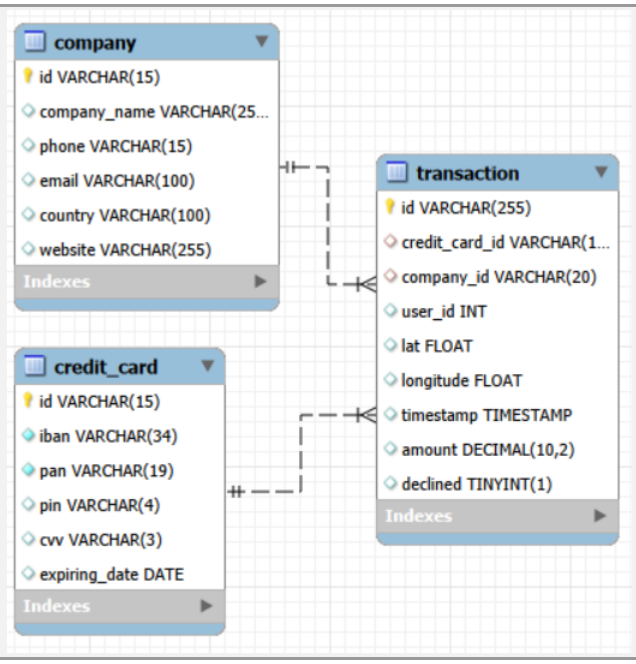
```sql
-- Anadimos la FK a credit_card, que no existia cuando se creo transaction.
ALTER TABLE `transaction`
    ADD CONSTRAINT `transaction_credit_card_fk`
    FOREIGN KEY (`credit_card_id`)
    REFERENCES `credit_card`(`id`)
    ON DELETE SET NULL;
```

Output

Action Output

#	Time	Action	Message
1	13:35:20	ALTER TABLE `transaction` ADD CONSTRAINT `transaction_credit_card_fk` FOREIGN KEY (`credit_card...	100000 row(s) affected Records: 100000 D

Diagrama entidad-relación



Comentarios sobre los resultados

Los datos de expiring_date en el archivo SQL estaban en un formato mm/dd/yy que no es compatible con el tipo Date de MySQL. Por eso he creado la tabla con este campo como VARCHAR y, tras importar los datos, los he convertido.

La nueva tabla está relacionada con transaction y, a través de esta, con company. No hay relación directa con company: en la tabla transaction se observa que la misma tarjeta de crédito es usada por distintas compañías.

Ejercicio 5 – Corrección del número de cuenta (CcU-2938)

Enunciado

Recursos Humanos ha identificado un error en el número de cuenta asociado a la tarjeta de crédito con ID CcU-2938. El valor correcto es TR323456312213576817699999. Muestra que el cambio se realizó.

Evidencia de ejecución (Workbench)

Antes

```
112  -- --- Ejercicio 5 ----------------------------------------------------
113  -- El departamento de Recursos Humanos ha identificado un error en el
114  -- número de cuenta asociado a la tarjeta de crédito con ID CcU-2938.
115  -- La información que debe mostrarse para este registro es:
116  -- TR323456312213576817699999. Recuerda mostrar que el cambio se realizó.
117  SELECT *
118  FROM `credit_card`
119  WHERE `id` = 'CcU-2938';
```

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell

id	iban	pan	pin	cvv	expiring_date
CcU-2938	TR301950312213576817638661	5424465566813633	3257	984	2022-10-30
NULL	NULL	NULL	NULL	NULL	NULL

credit_card 3

Output

Action Output

#	Time	Action
1	09:35:47	SELECT * FROM `credit_card` WHERE `id` = 'CcU-2938'

Después

```
117  UPDATE `credit_card`
118  SET `iban` = 'TR323456312213576817699999'
119  WHERE `id` = 'CcU-2938';
120
121  SELECT *
122  FROM `credit_card`
123  WHERE `id` = 'CcU-2938';
```

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cel

id	iban	pan	pin	cvv	expiring_date
CcU-2938	TR323456312213576817699999	5424465566813633	3257	984	2022-10-30
NULL	NULL	NULL	NULL	NULL	NULL

credit_card 5

Output

Action Output

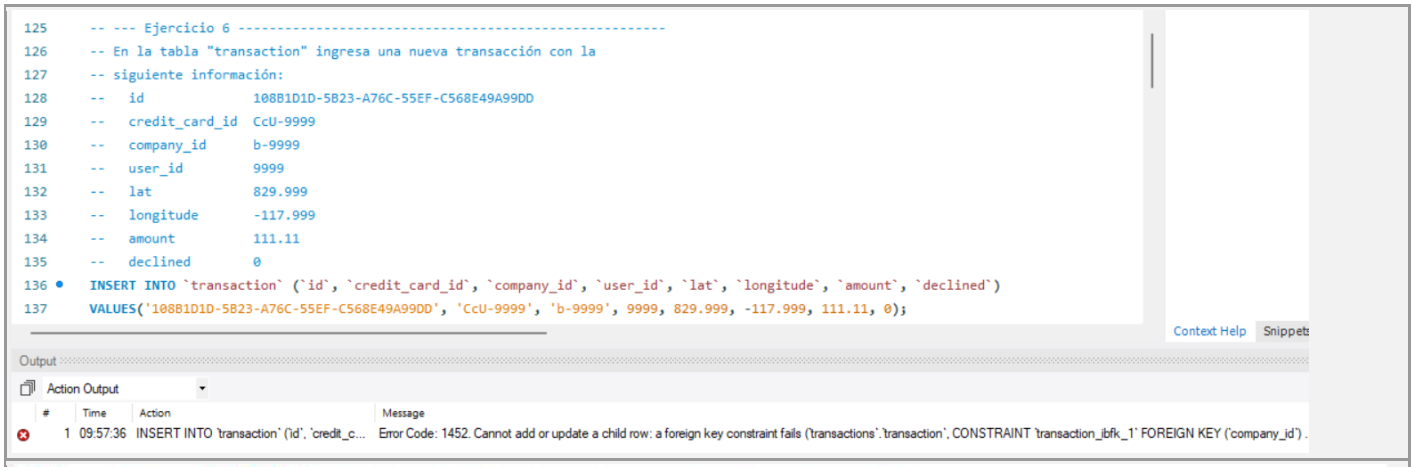
#	Time	Action
1	09:39:16	UPDATE `credit_card` SET `iban` = 'TR323456312213576817699999' WHERE `id` = 'CcU-2938'
2	09:39:16	SELECT * FROM `credit_card` WHERE `id` = 'CcU-2938'

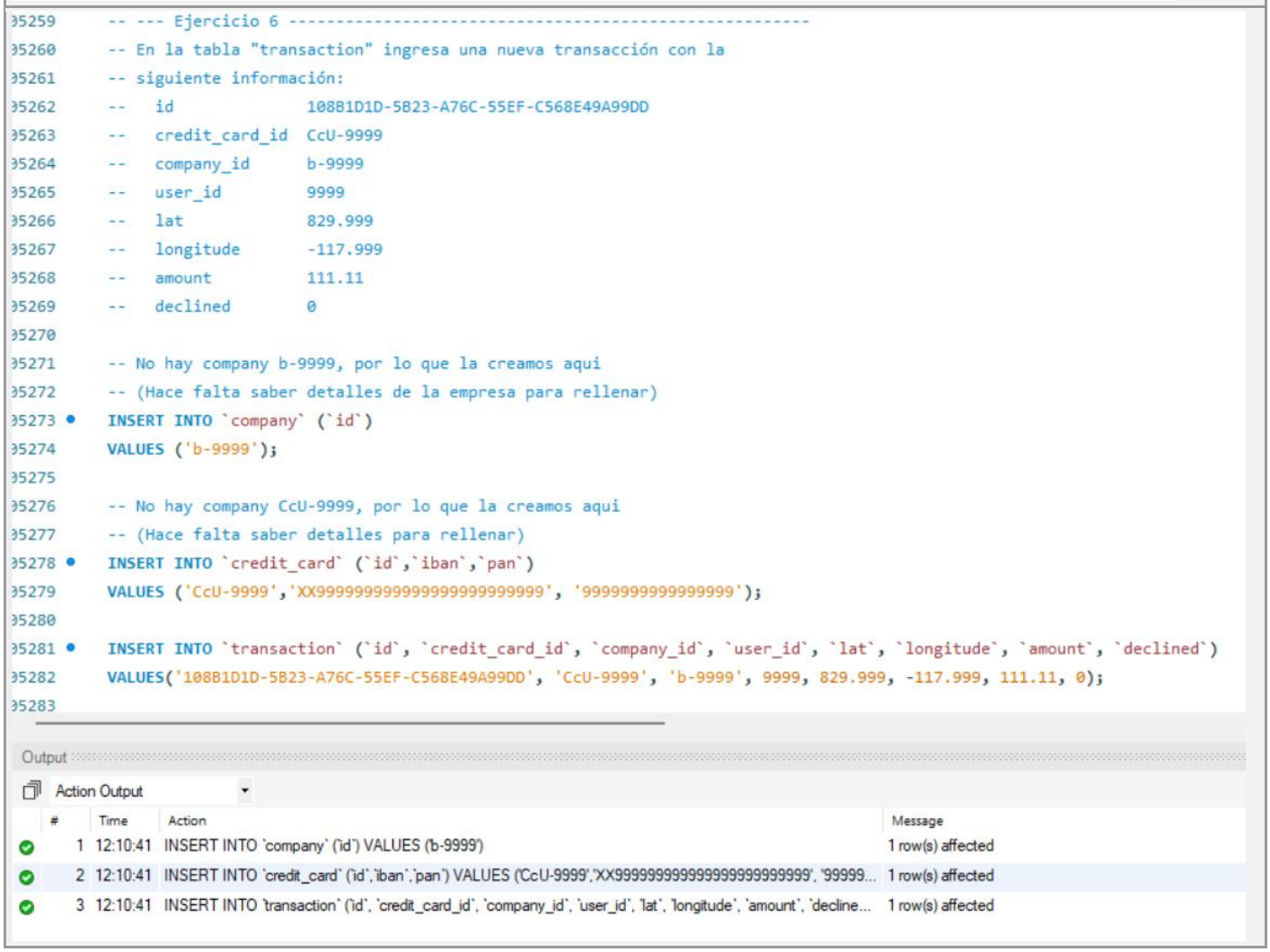
Ejercicio 6 – Inserción de una nueva transacción

Enunciado

En la tabla "transaction" ingresa una nueva transacción con: id 108B1D1D-5B23-A76C-55EF-C568E49A99DD, credit_card_id CcU-9999, company_id b-9999, user_id 9999, lat 829.999, longitude -117.999, amount 111.11, declined 0.

Evidencia de ejecución (Workbench)





Comentarios sobre los resultados

No es posible ingresar la transacción, pues no existe ninguna compañía con el id 'b-9999'. La restricción de clave foránea no lo permite.

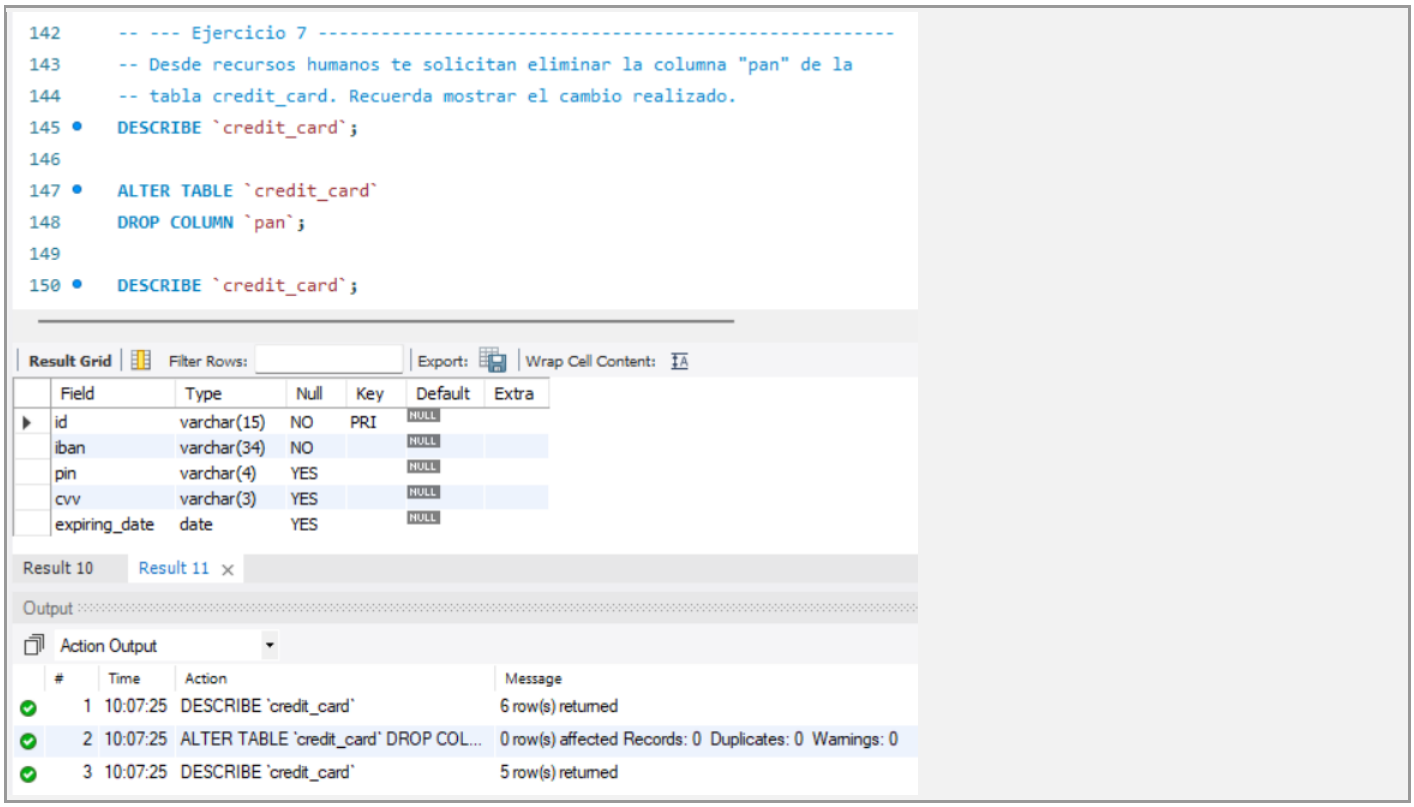
Para completar el ejercicio, he creado la empresa b-9999 y la tarjeta CcU-9999, pero no dispongo de los datos completos para crear correctamente.

Ejercicio 7 – Eliminación de la columna pan

Enunciado

Recursos Humanos solicita eliminar la columna "pan" de la tabla credit_card. Muestra el cambio realizado.

Evidencia de ejecución (Workbench)



```
142     -- --- Ejercicio 7 ----------------------------------------------------------
143     -- Desde recursos humanos te solicitan eliminar la columna "pan" de la
144     -- tabla credit_card. Recuerda mostrar el cambio realizado.
145     DESCRIBE `credit_card`;
146
147     ALTER TABLE `credit_card`
148     DROP COLUMN `pan`;
149
150     DESCRIBE `credit_card`;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

Field	Type	Null	Key	Default	Extra
id	varchar(15)	NO	PRI	NULL	
iban	varchar(34)	NO		NULL	
pin	varchar(4)	YES		NULL	
cvv	varchar(3)	YES		NULL	
expiring_date	date	YES		NULL	

Result 10 | Result 11

Output

Action Output

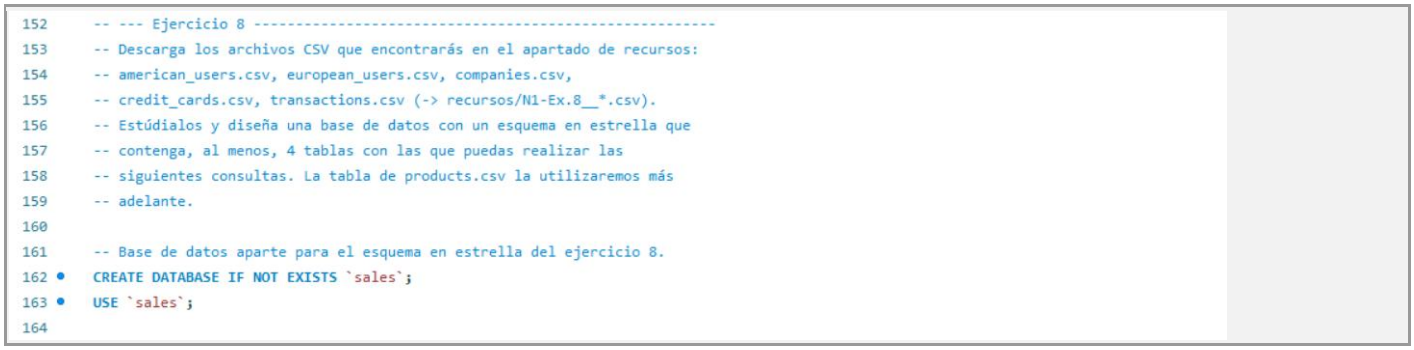
#	Time	Action	Message
1	10:07:25	DESCRIBE `credit_card`	6 row(s) returned
2	10:07:25	ALTER TABLE `credit_card` DROP COL...	0 row(s) affected Records: 0 Duplicates: 0 Warnings: 0
3	10:07:25	DESCRIBE `credit_card`	5 row(s) returned

Ejercicio 8 – Esquema en estrella a partir de los CSV

Enunciado

Descarga american_users.csv, european_users.csv, companies.csv, credit_cards.csv y transactions.csv. Estúdialos y diseña una base de datos con esquema en estrella con al menos 4 tablas. La tabla products.csv se usará más adelante.

Evidencia de ejecución (Workbench)



```
152     -- --- Ejercicio 8 ----------------------------------------------------------
153     -- Descarga los archivos CSV que encontrarás en el apartado de recursos:
154     -- american_users.csv, european_users.csv, companies.csv,
155     -- credit_cards.csv, transactions.csv (-> recursos/N1-Ex.8__*.csv).
156     -- Estúdialos y diseña una base de datos con un esquema en estrella que
157     -- contenga, al menos, 4 tablas con las que puedas realizar las
158     -- siguientes consultas. La tabla de products.csv la utilizaremos más
159     -- adelante.
160
161     -- Base de datos aparte para el esquema en estrella del ejercicio 8.
162     CREATE DATABASE IF NOT EXISTS `sales`;
163     USE `sales`;
164
```

```sql
    -- Todo VARCHAR de entrada para que el LOAD DATA no falle; cambiamos el tipo después.
    -- Dimensión de usuários
CREATE TABLE IF NOT EXISTS `users` (
    `id` VARCHAR(255) NULL,
    `name` VARCHAR(255) NULL,
    `surname` VARCHAR(255) NULL,
    `phone` VARCHAR(255) NULL,
    `email` VARCHAR(255) NULL,
    `birth_date` VARCHAR(255) NULL,
    `country` VARCHAR(255) NULL,
    `city` VARCHAR(255) NULL,
    `postal_code` VARCHAR(255) NULL,
    `address` VARCHAR(255) NULL,
    `signup_date` VARCHAR(255) NULL,
    `user_segment` VARCHAR(255) NULL,
    `income_band` VARCHAR(255) NULL,
    `region` VARCHAR(255) NULL
);

    -- Dimensión de empresas
CREATE TABLE IF NOT EXISTS `companies` (
    `company_id` VARCHAR(255) NULL,
    `company_name` VARCHAR(255) NULL,
    `phone` VARCHAR(255) NULL,
    `email` VARCHAR(255) NULL,
    `country` VARCHAR(255) NULL,
    `website` VARCHAR(255) NULL,
    `merchant_category` VARCHAR(255) NULL,
    `merchant_price_position` VARCHAR(255) NULL
);

    -- Dimensión de tarjetas
CREATE TABLE IF NOT EXISTS `credit_cards` (
    `id` VARCHAR(255) NULL,
    `user_id` VARCHAR(255) NULL,
    `iban` VARCHAR(255) NULL,
    `pan` VARCHAR(255) NULL,
    `pin` VARCHAR(255) NULL,
    `cvv` VARCHAR(255) NULL,
    `track1` VARCHAR(255) NULL,
    `track2` VARCHAR(255) NULL,
    `expiring_date` VARCHAR(255) NULL,
    `card_type` VARCHAR(255) NULL,
    `card_renewal_flag` VARCHAR(255) NULL
);

    -- Tabla de hechos: cada fila es una transacción
CREATE TABLE IF NOT EXISTS `transactions` (
    `id` VARCHAR(255) NULL,
    `card_id` VARCHAR(255) NULL,
    `business_id` VARCHAR(255) NULL,
    `timestamp` VARCHAR(255) NULL,
    `amount` VARCHAR(255) NULL,
    `declined` VARCHAR(255) NULL,
    `product_ids` VARCHAR(255) NULL,
    `user_id` VARCHAR(255) NULL,
    `lat` VARCHAR(255) NULL,
    `longitude` VARCHAR(255) NULL,
    `discount_amount` VARCHAR(255) NULL,
    `tax_amount` VARCHAR(255) NULL,
    `shipping_amount` VARCHAR(255) NULL,
    `channel` VARCHAR(255) NULL,
    `campaign_id` VARCHAR(255) NULL,
    `device_type` VARCHAR(255) NULL,
    `is_international` VARCHAR(255) NULL,
    `decline_reason` VARCHAR(255) NULL,
    `distance_km` VARCHAR(255) NULL
);

```

```sql
-- Carga los usuários de EE.UU.; region no viene en el CSV, la fijamos con SET region = America.
LOAD DATA
INFILE 'C:/ProgramData/MySQL/MySQL Server 8.0/Uploads/N1-Ex.8__american_users.csv'
INTO TABLE `users`
FIELDS TERMINATED BY ','
ENCLOSED BY '"'
IGNORE 1 ROWS
(`id`, `name`, `surname`, `phone`, `email`, `birth_date`, `country`, `city`, `postal_code`, `address`, `signup_date`, `user_segment`, `
SET `region` = 'America';

-- Carga los usuários de Europa, marcando region = Europe.
LOAD DATA
INFILE 'C:/ProgramData/MySQL/MySQL Server 8.0/Uploads/N1-Ex.8__european_users.csv'
INTO TABLE `users`
FIELDS TERMINATED BY ','
ENCLOSED BY '"'
IGNORE 1 ROWS
(`id`, `name`, `surname`, `phone`, `email`, `birth_date`, `country`, `city`, `postal_code`, `address`, `signup_date`, `user_segment`, `
SET `region` = 'Europe';

-- birth_date llega como 'Nov 17, 1985'; la convertimos antes de tipar la columna como DATE.
UPDATE `users`
SET `birth_date` = STR_TO_DATE(`birth_date`, '%b %e, %Y')
LIMIT 5000;

-- Tipos reales y PK una vez los datos ya están limpios.
ALTER TABLE `users`
    MODIFY `id` INT PRIMARY KEY,
    MODIFY `name` VARCHAR(255) NOT NULL,
    MODIFY `surname` VARCHAR(255) NOT NULL,
    MODIFY `birth_date` DATE,
    MODIFY `signup_date` DATE;

-- Carga las empresas
LOAD DATA
INFILE 'C:/ProgramData/MySQL/MySQL Server 8.0/Uploads/N1-Ex.8__companies.csv'
INTO TABLE `companies`
FIELDS TERMINATED BY ','
ENCLOSED BY '"'
IGNORE 1 ROWS
(`company_id`, `company_name`, `phone`, `email`, `country`, `website`, `merchant_category`, `merchant_price_position`);

-- company_id como PK; el resto se queda como texto.
ALTER TABLE `companies`
    MODIFY `company_id` VARCHAR(10) PRIMARY KEY,
    MODIFY `company_name` VARCHAR(255) NOT NULL;

-- Carga las tarjetas
LOAD DATA
INFILE 'C:/ProgramData/MySQL/MySQL Server 8.0/Uploads/N1-Ex.8__credit_cards.csv'
INTO TABLE `credit_cards`
FIELDS TERMINATED BY ','
ENCLOSED BY '"'
IGNORE 1 ROWS
(`id`, `user_id`, `iban`, `pan`, `pin`, `cvv`, `track1`, `track2`, `expiring_date`, `card_type`, `card_renewal_flag`);

-- formato de fecha (mm/dd/yy)
UPDATE `credit_cards`
SET `expiring_date` = STR_TO_DATE(`expiring_date`, '%m/%d/%y')
LIMIT 5000;

-- Tipamos user_id y expiring_date, y fijamos id como PK.
ALTER TABLE `credit_cards`
    MODIFY `id` VARCHAR(10) PRIMARY KEY,
    MODIFY `user_id` INT,
    MODIFY `expiring_date` DATE,
    MODIFY `card_renewal_flag` TINYINT;
```

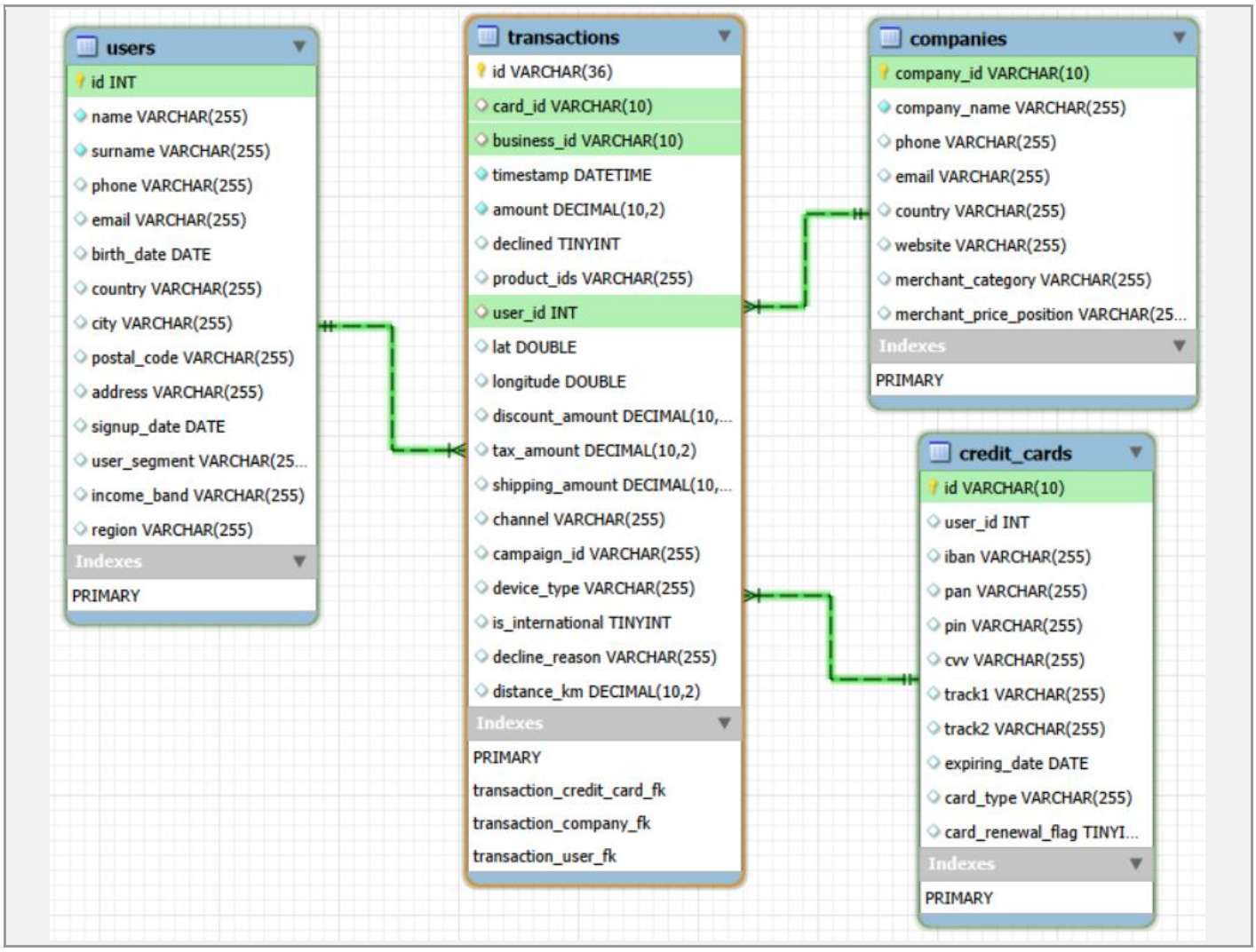
```
302     -- Diferente de los otros archivos, en este el separador es ';'
303 •   LOAD DATA
304     INFILE 'C:/ProgramData/MySQL/MySQL Server 8.0/Uploads/N1-Ex.8__transactions.csv'
305     INTO TABLE `transactions`
306     FIELDS TERMINATED BY ';'
307     ENCLOSED BY '"'
308     IGNORE 1 ROWS
309     (`id`, `card_id`, `business_id`, `timestamp`, `amount`, `declined`, `product_ids`, `user_id`, `lat`, `longitude`, `discount_amount`, `t
310
311     -- Tipamos la tabla de hechos
312 •   ALTER TABLE `transactions`
313         MODIFY `id` VARCHAR(36) PRIMARY KEY,
314         MODIFY `card_id` VARCHAR(10),
315         MODIFY `business_id` VARCHAR(10),
316         MODIFY `timestamp` DATETIME NOT NULL,
317         MODIFY `amount` DECIMAL(10,2) NOT NULL,
318         MODIFY `declined` TINYINT,
319         MODIFY `user_id` INT,
320         MODIFY `lat` DOUBLE,
321         MODIFY `longitude` DOUBLE,
322         MODIFY `discount_amount` DECIMAL(10,2),
323         MODIFY `tax_amount` DECIMAL(10,2),
324         MODIFY `shipping_amount` DECIMAL(10,2),
325         MODIFY `is_international` TINYINT,
326         MODIFY `distance_km` DECIMAL(10,2);
327
328     -- Conecta la tabla de hechos con la dimensión de tarjetas.
329 •   ALTER TABLE `transactions`
330         ADD CONSTRAINT `transaction_credit_card_fk`
331         FOREIGN KEY (`card_id`)
332         REFERENCES `credit_cards`(`id`)
333         ON DELETE SET NULL;
334
335     -- Conecta la tabla de hechos con la dimensión de empresas.
336 •   ALTER TABLE `transactions`
337         ADD CONSTRAINT `transaction_company_fk`
338         FOREIGN KEY (`business_id`)
339         REFERENCES `companies`(`company_id`)
340         ON DELETE SET NULL;
341
342     -- Conecta la tabla de hechos con la dimensión de usuarios.
343 •   ALTER TABLE `transactions`
344         ADD CONSTRAINT `transaction_user_fk`
345         FOREIGN KEY (`user_id`)
346         REFERENCES `users`(`id`)
347         ON DELETE SET NULL;
348
```

Output

Action Output

#	Time	Action	Message	Duration / Fetch
1	13:16:06	CREATE DATABASE IF NOT EXISTS `sales`	1 row(s) affected	0.000 sec
2	13:16:06	USE `sales`	0 row(s) affected	0.000 sec
3	13:16:06	CREATE TABLE IF NOT EXISTS `users` (`id` VARCHAR(255) NULL, `name` VARCHAR(255) NULL, `surnam...	0 row(s) affected	0.016 sec
4	13:16:06	CREATE TABLE IF NOT EXISTS `companies` (`company_id` VARCHAR(255) NULL, `company_name` VAR...	0 row(s) affected	0.031 sec
5	13:16:06	CREATE TABLE IF NOT EXISTS `credit_cards` (`id` VARCHAR(255) NULL, `user_id` VARCHAR(255) NULL...	0 row(s) affected	0.016 sec
6	13:16:06	CREATE TABLE IF NOT EXISTS `transactions` (`id` VARCHAR(255) NULL, `card_id` VARCHAR(255) NULL...	0 row(s) affected	0.015 sec
7	13:16:06	LOAD DATA INFILE 'C:/ProgramData/MySQL/MySQL Server 8.0/Uploads/N1-Ex.8__american_users.csv' I...	1010 row(s) affected Records: 1010 Deleted: 0 Skipped: 0 Warnings: 0	0.016 sec
8	13:16:07	LOAD DATA INFILE 'C:/ProgramData/MySQL/MySQL Server 8.0/Uploads/N1-Ex.8__european_users.csv' I...	3990 row(s) affected Records: 3990 Deleted: 0 Skipped: 0 Warnings: 0	0.047 sec
9	13:16:07	UPDATE `users` SET `birth_date` = STR_TO_DATE(`birth_date`, '%b %e, %Y') LIMIT 5000	5000 row(s) affected Rows matched: 5000 Changed: 5000 Warnings: 0	0.203 sec
10	13:16:07	ALTER TABLE `users` MODIFY `id` INT PRIMARY KEY, MODIFY `name` VARCHAR(255) NOT NULL, MO...	5000 row(s) affected Records: 5000 Duplicates: 0 Warnings: 0	0.141 sec
11	13:16:07	LOAD DATA INFILE 'C:/ProgramData/MySQL/MySQL Server 8.0/Uploads/N1-Ex.8__companies.csv' INTO ...	100 row(s) affected Records: 100 Deleted: 0 Skipped: 0 Warnings: 0	0.000 sec
12	13:16:07	ALTER TABLE `companies` MODIFY `company_id` VARCHAR(10) PRIMARY KEY, MODIFY `company_nam...	100 row(s) affected Records: 100 Duplicates: 0 Warnings: 0	0.047 sec
13	13:16:07	LOAD DATA INFILE 'C:/ProgramData/MySQL/MySQL Server 8.0/Uploads/N1-Ex.8__credit_cards.csv' INT...	5000 row(s) affected Records: 5000 Deleted: 0 Skipped: 0 Warnings: 0	0.062 sec
14	13:16:07	UPDATE `credit_cards` SET `expiring_date` = STR_TO_DATE(`expiring_date`, '%m/%d/%y') LIMIT 5000	5000 row(s) affected Rows matched: 5000 Changed: 5000 Warnings: 0	0.313 sec
15	13:16:07	ALTER TABLE `credit_cards` MODIFY `id` VARCHAR(10) PRIMARY KEY, MODIFY `user_id` INT, MODIFY `...	5000 row(s) affected Records: 5000 Duplicates: 0 Warnings: 0	0.141 sec
16	13:16:07	LOAD DATA INFILE 'C:/ProgramData/MySQL/MySQL Server 8.0/Uploads/N1-Ex.8__transactions.csv' INT...	100000 row(s) affected Records: 100000 Deleted: 0 Skipped: 0 Warnings: 0	3.656 sec
17	13:16:11	ALTER TABLE `transactions` MODIFY `id` VARCHAR(36) PRIMARY KEY, MODIFY `card_id` VARCHAR(10),...	100000 row(s) affected Records: 100000 Duplicates: 0 Warnings: 0	3.531 sec
18	13:16:15	ALTER TABLE `transactions` ADD CONSTRAINT `transaction_credit_card_fk` FOREIGN KEY (`card_id`) ...	100000 row(s) affected Records: 100000 Duplicates: 0 Warnings: 0	4.250 sec
19	13:16:19	ALTER TABLE `transactions` ADD CONSTRAINT `transaction_company_fk` FOREIGN KEY (`business_id`'...	100000 row(s) affected Records: 100000 Duplicates: 0 Warnings: 0	5.031 sec
20	13:16:24	ALTER TABLE `transactions` ADD CONSTRAINT `transaction_user_fk` FOREIGN KEY (`user_id`) REF...	100000 row(s) affected Records: 100000 Duplicates: 0 Warnings: 0	6.532 sec

Diagrama entidad-relación



Comentarios sobre los resultados

He creado una base de datos aparte (sales): aquí hay una tabla de hechos (transactions) y tres dimensiones (users, companies, credit_cards) en un modelo estrella.

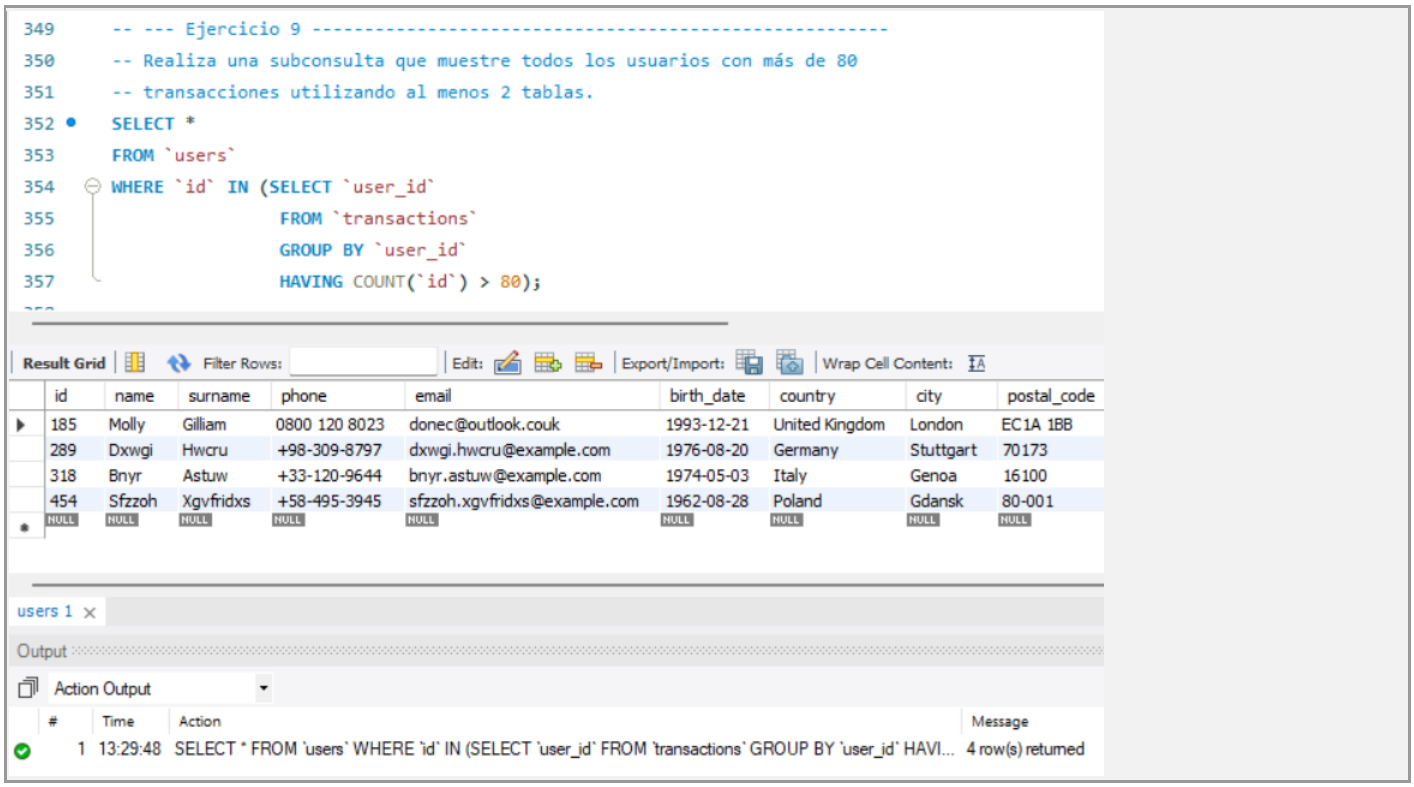
No he enlazado credit_cards con users mediante una FK directa: hacerlo conectaría dos dimensiones entre sí. La relación entre tarjeta y usuario ya queda cubierta indirectamente a través de la tabla de hechos (transactions.card_id y transactions.user_id).

Ejercicio 9 – Usuarios con más de 80 transacciones

Enunciado

Realiza una subconsulta que muestre todos los usuarios con más de 80 transacciones utilizando al menos 2 tablas.

Evidencia de ejecución (Workbench)



```
349     -- --- Ejercicio 9 ----------------------------------------------------------
350     -- Realiza una subconsulta que muestre todos los usuarios con más de 80
351     -- transacciones utilizando al menos 2 tablas.
352 •   SELECT *
353     FROM `users`
354     WHERE `id` IN (SELECT `user_id`
355                    FROM `transactions`
356                    GROUP BY `user_id`
357                    HAVING COUNT(`id`) > 80);
```

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content:

id	name	surname	phone	email	birth_date	country	city	postal_code
185	Molly	Gilliam	0800 120 8023	donec@outlook.couk	1993-12-21	United Kingdom	London	EC1A 1BB
289	Dxwgi	Hwcru	+98-309-8797	dxwgi.hwcru@example.com	1976-08-20	Germany	Stuttgart	70173
318	Bnyr	Astuw	+33-120-9644	bnyr.astuw@example.com	1974-05-03	Italy	Genoa	16100
454	Sfzzoh	Xgvfridxs	+58-495-3945	sfzzoh.xgvfridxs@example.com	1962-08-28	Poland	Gdansk	80-001
NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL

users 1

Output

Action Output

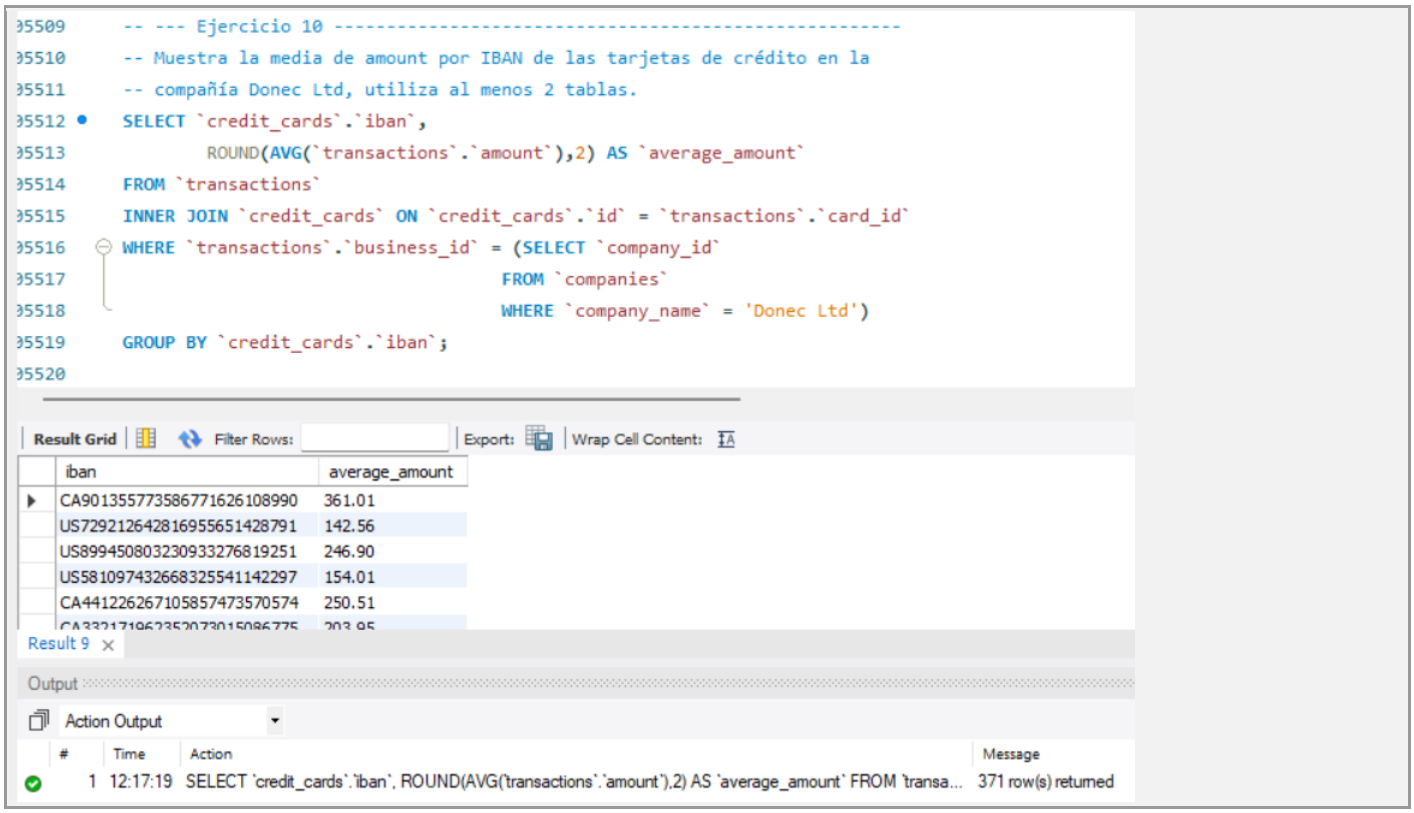
#	Time	Action	Message
1	13:29:48	SELECT * FROM `users` WHERE `id` IN (SELECT `user_id` FROM `transactions` GROUP BY `user_id` HAVI...	4 row(s) returned

Ejercicio 10 – Media de amount por IBAN (Donec Ltd)

Enunciado

Muestra la media de amount por IBAN de las tarjetas de crédito en la compañía Donec Ltd, utilizando al menos 2 tablas.

Evidencia de ejecución (Workbench)



```
95509     -- --- Ejercicio 10 ----------------------------------------------------------
95510     -- Muestra la media de amount por IBAN de las tarjetas de crédito en la
95511     -- compañía Donec Ltd, utiliza al menos 2 tablas.
95512 •   SELECT `credit_cards`.`iban`,
95513            ROUND(AVG(`transactions`.`amount`),2) AS `average_amount`
95514     FROM `transactions`
95515     INNER JOIN `credit_cards` ON `credit_cards`.`id` = `transactions`.`card_id`
95516     WHERE `transactions`.`business_id` = (SELECT `company_id`
95517                                           FROM `companies`
95518                                           WHERE `company_name` = 'Donec Ltd')
95519     GROUP BY `credit_cards`.`iban`;
95520
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

iban	average_amount
CA901355773586771626108990	361.01
US729212642816955651428791	142.56
US899450803230933276819251	246.90
US581097432668325541142297	154.01
CA441226267105857473570574	250.51
CA332171962352073015086775	203.95

Result 9

Output

Action Output

#	Time	Action	Message
1	12:17:19	SELECT `credit_cards`.`iban`, ROUND(AVG(`transactions`.`amount`),2) AS `average_amount` FROM `transa...	371 row(s) returned

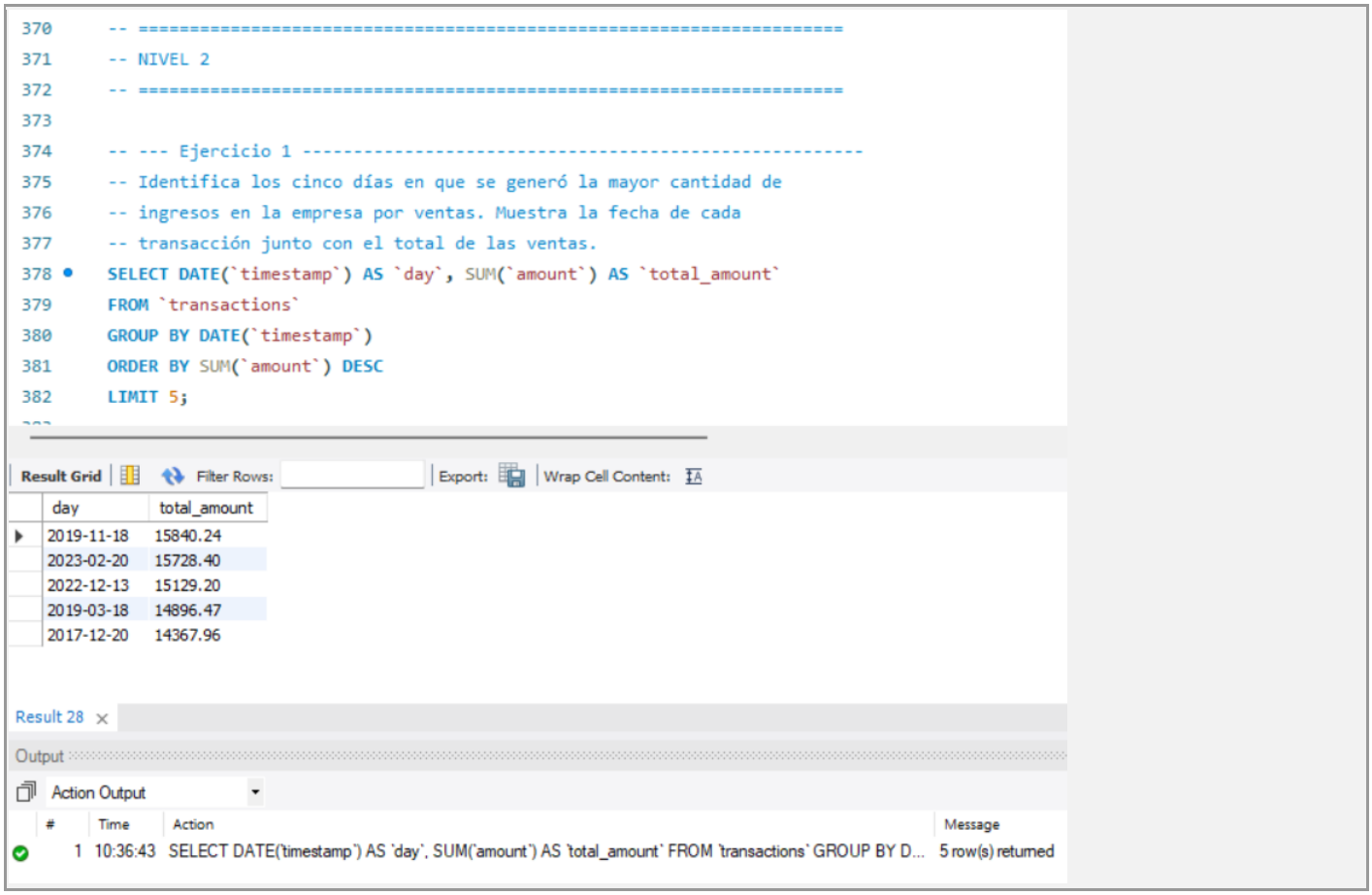
NIVEL 2

Ejercicio 1 – Top 5 días con mayores ingresos

Enunciado

Identifica los cinco días en que se generó la mayor cantidad de ingresos por ventas. Muestra la fecha de cada transacción junto con el total de las ventas.

Evidencia de ejecución (Workbench)



```
370     -- ========================================================================
371     -- NIVEL 2
372     -- ========================================================================
373
374     -- --- Ejercicio 1 ---------------------------------------------------------
375     -- Identifica los cinco días en que se generó la mayor cantidad de
376     -- ingresos en la empresa por ventas. Muestra la fecha de cada
377     -- transacción junto con el total de las ventas.
378     SELECT DATE(`timestamp`) AS `day`, SUM(`amount`) AS `total_amount`
379     FROM `transactions`
380     GROUP BY DATE(`timestamp`)
381     ORDER BY SUM(`amount`) DESC
382     LIMIT 5;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

day	total_amount
2019-11-18	15840.24
2023-02-20	15728.40
2022-12-13	15129.20
2019-03-18	14896.47
2017-12-20	14367.96

Result 28

Output

Action Output

#	Time	Action	Message
1	10:36:43	SELECT DATE(`timestamp`) AS `day`, SUM(`amount`) AS `total_amount` FROM `transactions` GROUP BY D...	5 row(s) returned

Ejercicio 2 – Transacciones entre 350 y 400 euros en fechas concretas

Enunciado

Presenta nombre, teléfono, país, fecha y amount de las empresas que realizaron transacciones entre 350 y 400 euros en el 29/04/2015, 20/07/2018 o 13/03/2024. Ordena de mayor a menor amount.

Evidencia de ejecución (Workbench)

```sql
-- --- Ejercicio 2 ------------------------------------------------------
-- Presenta el nombre, teléfono, país, fecha y amount, de aquellas
-- empresas que realizaron transacciones con un valor comprendido entre
-- 350 y 400 euros y en alguna de estas fechas: 29 de abril de 2015,
-- 20 de julio de 2018 y 13 de marzo de 2024. Ordena los resultados de
-- mayor a menor cantidad.
SELECT `companies`.`company_name`,
       `companies`.`phone`,
       `companies`.`country`,
       DATE(`transactions`.`timestamp`) AS `date`,
       `transactions`.`amount`
FROM `companies`
INNER JOIN `transactions` ON `transactions`.`business_id` = `companies`.`company_id`
WHERE `transactions`.`amount` BETWEEN 350 AND 400
    AND DATE(`transactions`.`timestamp`) IN ('2015-04-29', '2018-07-20', '2024-03-13')
ORDER BY `transactions`.`amount` DESC;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

company_name	phone	country	date	amount
Fringilla LLC	+1 150 784 1716	New Zealand	2015-04-29	390.33
Auctor Mauris Vel LLP	+32 133 447 8239	United States	2024-03-13	383.58
Mauris Institute	+75 473 619 9472	Sweden	2015-04-29	369.20
Pede Cum Ltd	+26 625 798 6126	Norway	2018-07-20	358.64
Netus Et Malesuada Ltd	+52 818 402 1638	Netherlands	2024-03-13	352.21
Tempor Diam Institute	+89 848 141 3136	Netherlands	2024-03-13	350.71

Result 66

Output

Action Output

#	Time	Action	Message
1	12:02:10	SELECT `companies`.`company_name`, `companies`.`phone`, `companies`.`country`, DATE(`trans...	6 row(s) returned

Ejercicio 3 – Empresas por umbral de 400 transacciones

Enunciado

Listado de empresas especificando si tienen igual o más de 400 transacciones, o menos.

Evidencia de ejecución (Workbench)

```
401     -- --- Ejercicio 3 ------------------------------------------------------
402     -- Necesitamos optimizar la asignación de recursos y dependerá de la
403     -- capacidad operativa que se requiera, por lo que te piden la
404     -- información sobre la cantidad de transacciones que realizan las
405     -- empresas, pero el departamento de recursos humanos es exigente y
406     -- quiere un listado de las empresas donde especifiques si tienen igual
407     -- o más de 400 transacciones o menos.
408     WITH `group_transactions` AS (
409             SELECT `transactions`.`business_id`,
410                     COUNT(`transactions`.`id`) AS `count_transactions`,
411                     (COUNT(`transactions`.`id`) >= 400) AS `exceed_capacity`
412             FROM `transactions`
413             GROUP BY `transactions`.`business_id`)
414     SELECT `companies`.`company_id`,
415             `companies`.`company_name`,
416             `group_transactions`.`count_transactions`,
417             `group_transactions`.`exceed_capacity`
418     FROM `companies`
419     LEFT JOIN `group_transactions` ON `group_transactions`.`business_id` = `companies`.`company_id`
420     ORDER BY `group_transactions`.`count_transactions`;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

company_id	company_name	count_transactions	exceed_capacity
b-2362	Lorem Eu Incorporated	380	0
b-2286	Fringilla LLC	397	0
b-2470	Nec Luctus LLC	399	0
b-2578	Dui Quis Institute	402	1
b-2446	Risus Associates	407	1
b-2226	Magna A Neque Industries	410	1
b-2526	Amet Luctus Vulputate Foundation	411	1
b-2298	Elit Etiam Laoreet Associates	418	1

Result 69

Output

Action Output

#	Time	Action	Message
1	12:05:23	WITH `group_transactions` AS (SELECT `transactions`.`business_id`, COUNT(`transactions`.`id`) AS `count_...	100 row(s) returned

Ejercicio 4 – Eliminación de una transacción

Enunciado

Elimina de la tabla transaction el registro con ID 000447FE-B650-4DCF-85DE-C7ED0EE1CAAD.

Evidencia de ejecución (Workbench)

```
418     -- --- Ejercicio 4 ------------------------------------------------------
419     -- Elimina de la tabla transaction el registro con ID
420     -- 000447FE-B650-4DCF-85DE-C7ED0EE1CAAD de la base de datos.
421 •   SELECT *
422     FROM `transactions`
423     WHERE `id` = '000447FE-B650-4DCF-85DE-C7ED0EE1CAAD';
424
425 •   DELETE FROM `transactions`
426     WHERE `id` = '000447FE-B650-4DCF-85DE-C7ED0EE1CAAD';
```

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content:

id	card_id	business_id	timestamp	amount	declined	product_ids	user_id	l
000447FE-B650-4DCF-85DE-C7ED0EE1CAAD	CcS-5019	b-2370	2016-12-21 20:07:18	172.56	0	66, 69, 87	438	4
NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NU

transactions 39 ×

Output

Action Output

#	Time	Action	Message
1	11:10:24	SELECT * FROM `transactions` WHERE `id` = '000447FE-B650-4DCF-85DE-C7ED0EE1CAAD'	1 row(s) returned
2	11:10:24	DELETE FROM `transactions` WHERE `id` = '000447FE-B650-4DCF-85DE-C7ED0EE1CAAD'	1 row(s) affected

Ejercicio 5 – Vista VistaMarketing

Enunciado

Crea una vista llamada VistaMarketing con: nombre de la compañía, teléfono de contacto, país de residencia y media de compra por compañía. Preséntala ordenada de mayor a menor media de compra.

Evidencia de ejecución (Workbench)

```
428     -- --- Ejercicio 5 ------------------------------------------------------
429     -- La sección de marketing desea tener acceso a información específica
430     -- para realizar análisis y estrategias efectivas. Se ha solicitado
431     -- crear una vista que proporcione detalles clave sobre las compañías y
432     -- sus transacciones. Será necesario que crees una vista llamada
433     -- VistaMarketing que contenga la siguiente información: nombre de la
434     -- compañía, teléfono de contacto, país de residencia, media de compra
435     -- realizada por cada compañía. Presenta la vista creada, ordenando los
436     -- datos de mayor a menor media de compra.
```

```
CREATE OR REPLACE VIEW `VistaMarketing` AS
WITH `group_transactions` AS (
        SELECT `transactions`.`business_id`,
                ROUND(AVG(`transactions`.`amount`),2) AS `average_amount`
        FROM `transactions`
        GROUP BY `transactions`.`business_id`)
SELECT `companies`.`company_name`,
        `companies`.`phone`,
        `companies`.`country`,
        `group_transactions`.`average_amount`
FROM `companies`
INNER JOIN `group_transactions` ON `group_transactions`.`business_id` = `companies`.`company_id`
ORDER BY `group_transactions`.`average_amount` DESC;

SELECT * FROM `VistaMarketing`
ORDER BY `average_amount` DESC;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

company_name	phone	country	average_amount
Ac Fermentum Incorporated	+64 601 244 6320	Germany	330.77
Ut Semper Foundation	+10 283 500 6449	Sweden	307.37
Orci Adipiscing Limited	+60 897 390 5820	United Kingdom	306.27
Dolor Vitae Limited	+6 421 567 9195	France	305.95
Nunc In Foundation	+13 267 741 2619	Italy	305.40
Maecenas Malesuada Fringilla Inc	+51 673 540 5289	Netherlands	305.15

VistaMarketing 17

Output

Action Output

#	Time	Action	Message
1	12:21:36	CREATE OR REPLACE VIEW `VistaMarketing` AS WITH `group_transactions` AS (SELECT `transactions`.`...	0 row(s) affected
2	12:21:36	SELECT * FROM `VistaMarketing` ORDER BY `average_amount` DESC	100 row(s) returned

NIVEL 3

Ejercicio 1 – Estado de las tarjetas de crédito (activa/inactiva)

Enunciado

Crea una tabla que refleje el estado de las tarjetas de crédito: inactiva si las tres últimas transacciones fueron declinadas, activa si al menos una no fue rechazada. Responde: cuantas tarjetas están activas.

Evidencia de ejecución (Workbench)

```
461  -- ========================================================================
462  -- NIVEL 3
463  -- ========================================================================
464
465  -- --- Ejercicio 1 ---------------------------------------------------------
466  -- Crea una nueva tabla que refleje el estado de las tarjetas de crédito
467  -- basado en si las tres últimas transacciones han sido declinadas
468  -- entonces está inactiva, si al menos una no ha sido rechazada entonces
469  -- está activa. Partiendo de esta tabla responde:
470  -- ¿Cuántas tarjetas están activas?
471  CREATE TABLE `credit_card_status` (
472      `card_id` VARCHAR(10),
473      `is_inactive` TINYINT,
474      PRIMARY KEY(`card_id`),
475      FOREIGN KEY (`card_id`) REFERENCES `credit_cards`(`id`)
476  );
477
478  INSERT INTO `credit_card_status` (`card_id`, `is_inactive`)
479  WITH `transaction_status` AS
480      (SELECT ROW_NUMBER() OVER (PARTITION BY `card_id` ORDER BY `timestamp` DESC) AS `row`,
481          `card_id`,
482          `declined`
483      FROM `transactions`)
484  SELECT `card_id`,
485      MIN(`declined`) AS `is_inactive`
486  FROM `transaction_status`
487  WHERE `row` <= 3
488  GROUP BY `card_id`
489  ON DUPLICATE KEY UPDATE `is_inactive` = `is_inactive`;
490
491  SELECT `is_inactive`,
492          COUNT(`card_id`) AS `count_cards`
493  FROM `credit_card_status`
494  GROUP BY `is_inactive`;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

is_inactive	count_cards
0	5000

Result 6

Output

Action Output

#	Time	Action	Message
1	12:44:08	CREATE TABLE `credit_card_status` (`card_id` VARCHAR(10), `is_inactive` TINYINT, PRIMARY KEY...	0 row(s) affected
2	12:44:08	INSERT INTO `credit_card_status` (`card_id`, `is_inactive`) WITH `transaction_status` AS (SELECT ROW_...	5000 row(s) affected Records: 5000 Duplicates: 0 Warnings: 0
3	12:44:09	SELECT `is_inactive`, COUNT(`card_id`) AS `count_cards` FROM `credit_card_status` GROUP BY `is_inacti...	1 row(s) returned

Comentarios sobre los resultados

Hay 0 tarjetas con las 3 últimas transacciones declinadas. Mirando solo las últimas transacciones de cada tarjeta, encontré un único caso con 2 rechazos. Todas las 5000 tarjetas están activas.

Ejercicio 2 – Ventas por producto (products.csv)

Enunciado

Crea una tabla que permita unir products.csv con la base de datos (transaction tiene product_ids). Consulta el número de veces que se ha vendido cada producto.

Evidencia de ejecución (Workbench)

```sql
-- --- Ejercicio 2 ----------------------------------------------------
-- Crea una tabla con la que podamos unir los datos del archivo
-- products.csv con la base de datos creada (ya que hasta ahora no
-- podíamos hacerlo), teniendo en cuenta que desde transaction tienes
-- product_ids. Genera la siguiente consulta:
-- Necesitamos conocer el número de veces que se ha vendido cada
--    producto.

-- Nueva dimensión: productos.
CREATE TABLE `products` (
    `id` VARCHAR(255),
    `product_name` VARCHAR(255),
    `price` VARCHAR(255),
    `colour` VARCHAR(255),
    `weight` VARCHAR(255),
    `warehouse_id` VARCHAR(255),
    `category` VARCHAR(255),
    `brand` VARCHAR(255),
    `cost` VARCHAR(255),
    `launch_date` VARCHAR(255)
);

-- Carga los productos
LOAD DATA
INFILE 'C:/ProgramData/MySQL/MySQL Server 8.0/Uploads/N1-Ex.8__products.csv'
INTO TABLE `products`
FIELDS TERMINATED BY ','
ENCLOSED BY '"'
IGNORE 1 ROWS
(`id`, `product_name`, `price`, `colour`, `weight`, `warehouse_id`, `category`, `brand`, `cost`, `launch_date`);

-- Los precios tienen '$' en el archivo de carga.
UPDATE `products`
SET `price` = REPLACE(`price`, '$', ''),
    `cost` = REPLACE(`cost`, '$', '')
LIMIT 1000;

-- Tipado de la tabla products.
ALTER TABLE `products`
    MODIFY `id` INT PRIMARY KEY,
    MODIFY `price` DECIMAL(10,2),
    MODIFY `weight` DECIMAL(5,2),
    MODIFY `cost` DECIMAL(10,2),
    MODIFY `launch_date` DATE;

-- Tabla puente para la relación N:M entre transactions y products.
CREATE TABLE `product_transaction` (
    `transaction_id` VARCHAR(36),
    `product_id` INT,
    PRIMARY KEY (`transaction_id`, `product_id`),
    FOREIGN KEY (`transaction_id`) REFERENCES `transactions`(`id`),
    FOREIGN KEY (`product_id`) REFERENCES `products`(`id`)
);

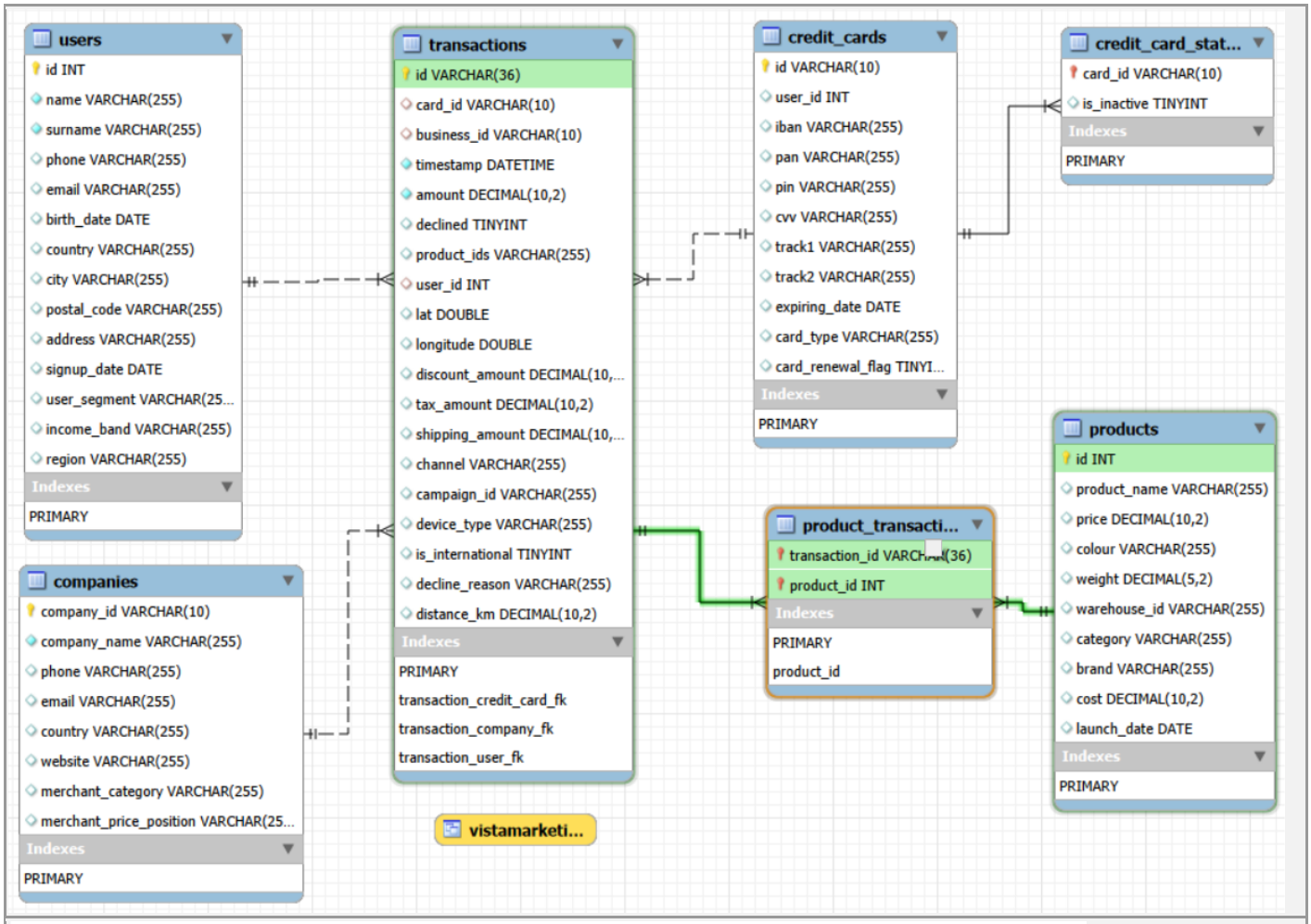
```

```sql
-- product_ids es una lista separada por comas; el CTE recursivo la
-- separa en una fila por producto.
INSERT INTO `product_transaction` (`transaction_id`,`product_id`)
WITH RECURSIVE `product_transactions` AS (
        SELECT `id`,
                TRIM(LEFT(`product_ids`, LOCATE(',', `product_ids`) - 1)) AS `product_id`,
                TRIM(RIGHT(`product_ids`, CHAR_LENGTH(`product_ids`) - LOCATE(',', `product_ids`))) AS `remaining_ids`
        FROM `transactions`
        WHERE LOCATE(',', `product_ids`) > 0
    UNION ALL
        SELECT `id`,
                TRIM(`product_ids`),
                NULL
        FROM `transactions`
        WHERE LOCATE(',', `product_ids`) = 0
    UNION ALL
        SELECT `id`,
                TRIM(LEFT(`remaining_ids`, LOCATE(',', `remaining_ids`) - 1)),
                TRIM(RIGHT(`remaining_ids`, CHAR_LENGTH(`remaining_ids`) - LOCATE(',', `remaining_ids`)))
        FROM `product_transactions`
        WHERE `remaining_ids` IS NOT NULL AND LOCATE(',', `remaining_ids`) > 0
    UNION ALL
        SELECT `id`,
                `remaining_ids`,
                NULL
        FROM `product_transactions`
        WHERE `remaining_ids` IS NOT NULL AND LOCATE(',', `remaining_ids`) = 0
)
SELECT `id`, `product_id` FROM `product_transactions` ORDER BY `id`;
```

Output

Action Output

#	Time	Action	Message
1	16:59:02	CREATE TABLE 'products' ('id' VARCHAR(255), 'product_name' VARCHAR(255), 'price' VARCHAR(255), '...	0 row(s) affected
2	16:59:02	LOAD DATA INFILE 'C:/ProgramData/MySQL/MySQL Server 8.0/Uploads/N1-Ex.8__products.csv' INTO T...	100 row(s) affected Records: 100 Deleted: 0 Skipped: 0 Warnings: 0
3	16:59:02	UPDATE 'products' SET 'price' = REPLACE('price', '$', ''), 'cost' = REPLACE('cost', '$', '') LIMIT 1000	100 row(s) affected Rows matched: 100 Changed: 100 Warnings: 0
4	16:59:02	ALTER TABLE 'products' MODIFY 'id' INT PRIMARY KEY, MODIFY 'price' DECIMAL(10,2), MODIFY 'weig...	100 row(s) affected Records: 100 Duplicates: 0 Warnings: 0
5	16:59:02	CREATE TABLE 'product_transaction' ('transaction_id' VARCHAR(36), 'product_id' INT, PRIMARY KEY ('tr...	0 row(s) affected
6	16:59:02	INSERT INTO 'product_transaction' ('transaction_id','product_id') WITH RECURSIVE 'product_transactions...	253388 row(s) affected Records: 253388 Duplicates: 0 Warnings: 0




```
-- Ventas por producto
-- LEFT JOIN para incluir productos con 0 ventas.
SELECT `products`.`id`,
       `products`.`product_name`,
       COUNT(`transactions`.`id`) AS `count_sales`
FROM `products`
LEFT JOIN `product_transaction` ON `product_transaction`.`product_id` = `products`.`id`
LEFT JOIN `transactions` ON `transactions`.`id` = `product_transaction`.`transaction_id`
                        AND `transactions`.`declined` = 0
GROUP BY `products`.`id`
ORDER BY COUNT(`transactions`.`id`);
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

id	product_name	count_sales
43	duel	2391
42	sith	2404
19	dooku solo	2423
22	chewbacca mustafar	2428
71	Tully Dorne	2437

Result 28

Output

Action Output

#	Time	Action	Message
1	12:41:51	SELECT `products`.`id`, `products`.`product_name`, COUNT(`transactions`.`id`) AS `count_sales` FROM `pro...	100 row(s) returned

Comentarios sobre los resultados

He creado una nueva tabla ‘product_transaction’ con **dos claves foráneas** para conectar ‘products’ con ‘transactions’. Separé product_ids de la columna ‘product_ids’ de la tabla `transactions` con un CTE recursivo (una fila por producto y transacción) en la tabla ‘product_transaction’.

Los 100 productos tienen ventas.